

Lesson 1: The Core Concept of Paging and Address Translation Mechanics

Slide Range: Slide 1 to Slide 7[cite: 5]

The Big Picture: Why Do We Need Paging? (Slides 1–3)

In the previous chapter, we saw that **Segmentation** solved the massive internal fragmentation of pure Base & Bounds by carving memory into logical segments: Code, Heap, and Stack[cite: 4, 5].

However, segmentation introduced a fatal flaw: **External Fragmentation**[cite: 4, 5].

- Because segments have **variable sizes** (e.g., Code might be 2 KB, Heap might be 7 KB, Stack might be 3 KB), physical memory quickly gets chopped into scattered, awkward little gaps[cite: 4, 5].
- You might have 24 KB of total free RAM, but if it is split into three separate 8 KB holes, a request for a 20 KB segment will fail[cite: 4]!
- Fixing that required **Compaction**—pausing execution and copying megabytes of memory across the bus, which kills system performance[cite: 4].

The fundamental problem with segmentation is **variable sizing**[cite: 5].

So, system architects asked:

"What if we stop chopping memory into variable-sized logical segments, and instead chop EVERYTHING into uniform, fixed-sized blocks?"[cite: 5]

This is the birth of **Paging**[cite: 5]:

The Two Halves of Paging	
Virtual Address Space is chopped into fixed units called:	Physical Memory (RAM) is chopped into fixed slots called:
PAGES (e.g., 4 KB each)	PAGE FRAMES (PFN) (Identical 4 KB size!)

1. **Pages:** The process’s virtual address space is sliced into fixed-sized blocks called **pages**[cite: 5].
2. **Page Frames (Physical Frames):** Real physical RAM is divided into slots of the exact same size, called **page frames**[cite: 5].
3. **The Page Table:** Because a page can be placed into *any* free physical page frame anywhere in RAM, each process maintains a translation map called a **Page Table** to keep track of where every

virtual page currently lives[cite: 5].

The Advantages of Paging: Flexibility and Simplicity (Slide 3)

- **No External Fragmentation:** Because every allocation request is for a uniform, fixed-sized block (e.g., 4 KB or 16 bytes in simple examples), every free slot in RAM fits any incoming page perfectly[cite: 5]. There are no awkward variable-sized holes left behind[cite: 4, 5].
- **Simplicity in Free-Space Management:** The OS free list simply tracks a list of empty frame numbers: [Frame 1, Frame 4, Frame 6] [cite: 5]. If a process needs memory, the OS hands it any available frame from the list[cite: 5].
- **Flexibility:** The OS does not need to assume or care which direction the heap grows or how the stack shrinks[cite: 5]. It just allocates pages as needed[cite: 5].

Walking Through a Toy Example (Slides 4 & 5)

Let us examine the concrete example from Slide 5[cite: 5]:

- **Total Physical Memory:** 128 bytes, partitioned into 16-byte frames[cite: 5].
Number of frames = $\frac{128 \text{ bytes}}{16 \text{ bytes}} = 8 \text{ page frames (Frame 0 to Frame 7)}$ [cite: 5]
- **Process Address Space:** 64 bytes total, partitioned into 16-byte pages[cite: 5].
Number of virtual pages = $\frac{64 \text{ bytes}}{16 \text{ bytes}} = 4 \text{ virtual pages (Page 0 to Page 3)}$ [cite: 5]

Process Virtual View (64 Bytes)	Physical RAM (128 Bytes)
0 +-----+ Page 0 of AS	0 +-----+ Page Frame 0 (Reserved for OS)
16 +-----+ Page 1 of AS	16 +-----+ Page Frame 1 (Unused)
32 +-----+ Page 2 of AS	32 +-----+ Page Frame 2 -> Holds Virtual Page 3
48 +-----+ Page 3 of AS	48 +-----+ Page Frame 3 -> Holds Virtual Page 0
64 +-----+	64 +-----+ Page Frame 4 (Unused)
	80 +-----+ Page Frame 5 -> Holds Virtual Page 2
	96 +-----+ Page Frame 6 (Unused)
	112 +-----+ Page Frame 7 -> Holds Virtual Page 1
	128 +-----+

Notice what just happened:

- Virtual Page 0 is placed in Physical Frame 3[cite: 5].
- Virtual Page 1 is placed in Physical Frame 7[cite: 5].
- Virtual Page 2 is placed in Physical Frame 5[cite: 5].

- Virtual Page 3 is placed in Physical Frame 2[cite: 5].

The pages of the address space are scattered non-contiguously throughout physical RAM[cite: 5]. To make sense of this scatter, the OS builds a **Page Table** (a lookup dictionary)[cite: 5]:

Virtual Page Number (VPN)	Physical Frame Number (PFN)
0	3
1	7
2	5
3	2

Anatomy of a Virtual Address: VPN and Offset (Slide 6 & 7)

How does the hardware translate a virtual address into a physical address[cite: 5]?

Any virtual address is split by the hardware into two distinct parts[cite: 5]:

1. **VPN (Virtual Page Number):** Selects *which page* of the address space is being accessed[cite: 5].
2. **Offset:** Selects *which exact byte* within that page is desired[cite: 5].

The Bit Breakdown for Our 64-Byte Address Space:

- Total address space size = 64 bytes = $2^6 \implies$ **6 bits total** (named **Va5** through **Va0**)[cite: 5].
- Page size = 16 bytes = $2^4 \implies$ **4 bits for the Offset**[cite: 5].
- Remaining bits for VPN = $6 - 4 = 2$ bits for the VPN ($2^2 = 4$ pages)[cite: 5].

6-Bit Virtual Address Structure:					
5	4	3	2	1	0
Va5	Va4	Va3	Va2	Va1	Va0
VPN		Offset			
(2 bits)		(4 bits)			

Let's Translate Virtual Address 21 (Slide 6 & 7):

Suppose the CPU executes an instruction targeting virtual address **21**[cite: 5].

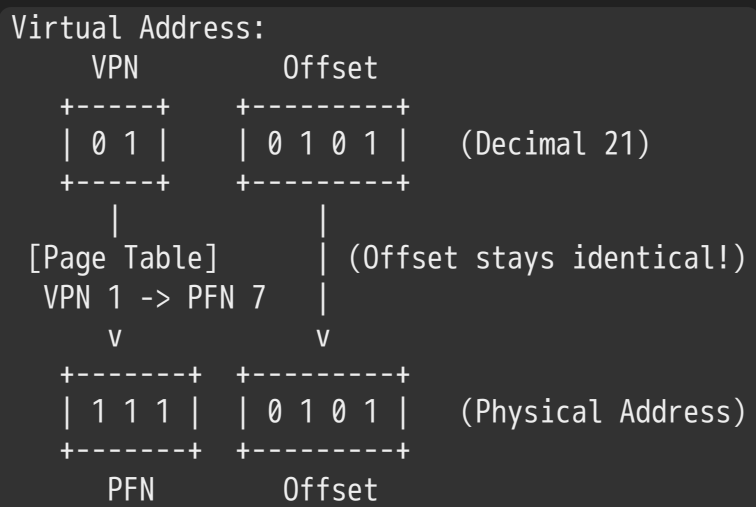
1. Convert **21** into a 6-bit binary number[cite: 5]: $21_{10} = 010101_2$ [cite: 5]
2. Split the bits into VPN and Offset[cite: 5]:
 - **VPN (top 2 bits):** **01** in binary = **1** in decimal[cite: 5]. This means the data lives on **Virtual Page 1**[cite: 5].
 - **Offset (lower 4 bits):** **0101** in binary = **5** in decimal[cite: 5]. This means we want byte index 5 inside that page[cite: 5].

3. Look up the VPN in the Page Table[cite: 5]:

- Index into entry 1: **VPN 1 maps to PFN 7** (**111** in binary)[cite: 5].

4. Form the Physical Address[cite: 5]:

- Replace the 2-bit VPN (**01**) with the 3-bit Physical Frame Number (**111**)[cite: 5].
- **Keep the 4-bit offset identical (**0101**)!** (The position of the byte inside the page does not change)[cite: 5].



Converting the resulting binary 1110101_2 to decimal[cite: 5]: $1110101_2 = (1 \times 64) + (1 \times 32) + (1 \times 16) + (0 \times 8) + (1 \times 4) + (0 \times 2) + (1 \times 1) = 64 + 32 + 16 + 4 + 1 = \mathbf{117}$ [cite: 5]

Or, using simple frame arithmetic: $\text{Physical Address} = (\text{PFN} \times \text{Page Size}) + \text{Offset} = (7 \times 16) + 5 = 112 + 5 = \mathbf{117}$ [cite: 5]

The hardware reads physical byte 117 directly from RAM[cite: 5].

Let me know if this translation flow is crystal clear, and we will move to **Lesson 2 (Slides 8 to 12)** to look at how large page tables get, where they are stored, what flags exist inside a Page Table Entry (PTE), and the real x86 PTE format[cite: 5]!

For an additional step-by-step lecture walkthrough of these mechanisms, review this [Operating Systems Paging Introduction video](http://googleusercontent.com/youtube_content/1), which explains the fundamentals of page frames, page tables, and address translation. http://googleusercontent.com/youtube_content/1

2026-09-03 13:18:03

You 

next

Lesson 2: Page Table Scaling, Memory Placement, and Page Table Entry (PTE) Flags

Slide Range: Slide 8 to Slide 12[cite: 5]

The Space Explosion: Where Are Page Tables Stored? (Slide 8)

In the toy example from Lesson 1, the page table had only 4 entries[cite: 5]. But what happens in a real-world 32-bit operating system[cite: 5]?

Let's do the math step-by-step[cite: 5]:

- **Virtual Address Space:** 32 bits, meaning 2^{32} bytes = 4 GB of addressable space[cite: 5].
- **Standard Page Size:** 4 KB = 4096 bytes = 2^{12} bytes[cite: 5].
- **Offset Bits:** The offset requires 12 bits to address every byte inside a 4 KB page ($2^{12} = 4096$) [cite: 5].
- **VPN Bits:** The remaining bits form the Virtual Page Number[cite: 5]: 32 bits total – 12 offset bits = 20 bits for the VPN[cite: 5]
- **Total Number of Virtual Pages:** $2^{20} = 1,048,576$ pages per process[cite: 5]

32-Bit Virtual Address Breakdown:		
31	12 11	0
Virtual Page Number (VPN)		Offset in Page
(20 bits)		(12 bits)
1,048,576 Pages		4,096 Bytes

If each **Page Table Entry (PTE)** consumes **4 bytes** (to store the physical frame number and control flags), how large is the page table for **one single process**[cite: 5]?

Page Table Size = 2^{20} entries $\times$ 4 bytes per entry = 4,194,304 bytes = **4 MB**[cite: 5]

- **Can we store a 4 MB page table in on-chip CPU registers?**
No[cite: 5]! CPU register files hold a few dozen registers (bytes, not megabytes)[cite: 5].
- **What if you run 100 processes?**
 $100 \text{ processes} \times 4 \text{ MB} = \mathbf{400 \text{ MB}}$ of RAM just to track page mappings!

The Big Takeaway: Page tables are far too massive to fit inside the CPU[cite: 5]. Therefore, **page tables themselves must be stored in physical memory (RAM)**, allocated and managed within the OS kernel's memory space[cite: 5].

The Page Table as an In-Memory Array (Slide 9 & 10)

Slide 9 shows how the OS places a process's page table in physical memory[cite: 5]:

Physical Memory

0	+-----+ Page Frame 0: Process's PAGE TABLE <--- Holds array: [PTE 0, PTE 1, PTE 2, PTE 3][cite: 5]
16	+-----+ Page Frame 1: (Unused)
32	+-----+ Page Frame 2: Page 3 of AS
48	+-----+ Page Frame 3: Page 0 of AS
64	+-----+ Page Frame 4: (Unused)
80	+-----+ Page Frame 5: Page 2 of AS
96	+-----+ Page Frame 6: (Unused)
112	+-----+ Page Frame 7: Page 1 of AS
128	+-----+

- **Linear Page Table Structure:** In its simplest form, a page table is just a flat array indexed directly by the **VPN**[cite: 5].
- **Contiguity Requirement:** Because the CPU uses simple base-index addressing, the array entries of a linear page table must be contiguous in physical memory[cite: 5].
- The CPU keeps a dedicated hardware pointer called the **Page Table Base Register (PTBR)**, which stores the physical starting address of the active process's page table[cite: 5].
- To find any PTE, the hardware simply evaluates[cite: 5]: $\text{Address of PTE} = \text{PTBR} + (\text{VPN} \times \text{sizeof(PTE)})$ [cite: 5]

Inside a Page Table Entry: Common Flags (Slide 11)

A Page Table Entry (PTE) stores more than just the Physical Frame Number (PFN)[cite: 5]. It contains essential control and status bits:

Common Flags in a PTE	
Valid Bit	1 = The virtual page is valid and in use by process. 0 = Unused/unallocated page (access causes segfault).
Protect Bits	Access rights: Read (R), Write (W), and/or Execute (X).
Present Bit	1 = The page resides currently in physical RAM. 0 = The page has been swapped out to disk.
Dirty Bit	1 = Page was modified (written to) since loaded in RAM. 0 = Page is clean (matches disk copy).
Reference Bit (Accessed)	1 = Page was recently accessed (read or written). Used by OS page replacement policies (e.g., LRU).

Deep Dive on Crucial Flags (Slide 11 Annotations):

1. Valid Bit vs. Segmentation Fault:

In a 4 GB space, a program might only use a few pages for code, a few for heap, and a few for stack[cite: 4, 5]. All the empty space in between is marked `Valid = 0` [cite: 5]. If the program tries to dereference a pointer in that empty space, the MMU sees `Valid == 0` and raises an exception (Segmentation Fault)[cite: 5].

2. Present Bit vs. Swapping:

Suppose 4 processes need 12 pages combined, but RAM only has 8 physical frames available[cite: 5]. The OS evicts less active pages out to the hard disk[cite: 5]. When a page is pushed to disk, the OS clears its `Present Bit` to `0` [cite: 5]. If the process later accesses that page, the MMU triggers a **Page Fault**, prompting the OS to read it back from disk into RAM[cite: 5].

3. Dirty Bit vs. Disk Writes:

If a page was swapped into RAM, read from, but never modified (`Dirty == 0`), the OS can discard it when evicting it because the disk already holds an identical copy[cite: 5]. But if `Dirty == 1` , the process changed its contents, so the OS must write the page back to disk before reusing the frame[cite: 5]!

Real-World Hardware: The x86 Page Table Entry (Slide 12)

Slide 12 shows the exact layout of a 32-bit entry on Intel x86 architectures[cite: 5]:

31	12	11	10	9	8	7	6	5	4	3	2	1	0
Physical Frame Number (PFN) (20 bits)												G	PAT
												D	A
												PCD	PWT
												U	W
												/S	/R
												P	

- **Bits 31–12 (20 bits): The Physical Frame Number (PFN)**[cite: 5]. When combined with the 12-bit offset, this builds the complete 32-bit physical RAM address[cite: 5].
- **Bit 0 (P - Present):** 1 if the page is currently in RAM, 0 if swapped to disk[cite: 5].
- **Bit 1 (W/R - Read/Write):** 1 allows write operations; 0 enforces read-only access[cite: 5].
- **Bit 2 (U/S - User/Supervisor):** 1 allows user-mode programs to access the page; 0 restricts access to kernel mode only (protecting OS code)[cite: 5].
- **Bit 3 (PWT - Page-level Write-Through):** Controls cache write-through policy[cite: 5].
- **Bit 4 (PCD - Page-level Cache Disable):** Disables CPU caching for this page (used for memory-mapped I/O devices)[cite: 5].
- **Bit 5 (A - Accessed):** Set by hardware whenever the page is read or written[cite: 5].
- **Bit 6 (D - Dirty):** Set by hardware whenever a write operation touches the page[cite: 5].
- **Bit 7 (PAT - Page Attribute Table) & Bit 8 (G - Global):** Prevents the translation from being flushed during a context switch if marked global[cite: 5].

Let me know if this breakdown of page tables and PTE flags makes sense, and we will move to **Lesson 3 (Slides 13 to 17)** to examine why simple paging cuts memory performance in half, trace memory references in assembly, and see how hardware introduces the Translation Lookaside Buffer (TLB) to fix it[cite: 5]!

2026-09-03 13:18:35

You 

next

 Gemini 3.8 Flash

Lesson 3: Paging Performance Overhead, Hardware Translation Flow, and Memory Tracing

Slide Range: Slide 13 to Slide 17[cite: 5]

The Core Problem: Why Simple Paging is Slow (Slide 13)

In Lesson 2, we saw that page tables are too large to fit inside CPU registers, so they must live directly inside physical RAM[cite: 5].

This creates a severe performance bottleneck[cite: 5]:

Without Paging (Direct Physical Access):

```
[ CPU ] -----> [ Physical RAM ]  
1 Memory Access = 100 ns
```

With Pure Paging (Page Table in RAM):

```
[ CPU ] -----> [ Step 1: Read PTE from RAM ] -----> [ Physical RAM ]  
                  (Wait 100 ns)  
                  |  
                  v  
[ Step 2: Read Actual Data from RAM ] -----> [ Physical RAM ]  
                  (Wait 100 ns)
```

To complete a single memory reference (such as loading a variable), the hardware must[cite: 5]:

1. Perform **one extra memory access** to fetch the Page Table Entry (PTE) from RAM[cite: 5].
2. Perform the **actual memory access** to read or write the data at the translated physical address[cite: 5].

Every single memory reference now requires **at least two physical RAM accesses**[cite: 5]. If accessing physical RAM takes 100 ns, your program's memory performance is cut in half (200 ns per access) [cite: 5]!

Hardware Control Flow: How the MMU Translates an Address (Slides 14 & 15)

Slides 14 and 15 provide the exact hardware pseudocode executed on every virtual memory reference[cite: 5].

Step 1: Locating and Fetching the PTE (Slide 14)

```
c  
  
// 1. Extract the VPN from the virtual address  
VPN = (VirtualAddress & VPN_MASK) >> SHIFT; //[cite: 5]  
  
// 2. Form the physical address of the Page Table Entry (PTE)  
PTEAddr = PTBR + (VPN * sizeof(PTE)); //[cite: 5]  
  
// 3. Fetch the PTE from RAM (THIS IS THE SLOW EXTRA MEMORY ACCESS!)  
PTE = AccessMemory(PTEAddr); //[cite: 5]
```

- **PTBR (Page Table Base Register)**: Holds the physical starting address of the running process's page table in RAM[cite: 5].
- The MMU multiplies the **VPN** by the size of each entry (e.g., 4 bytes) and adds that offset to **PTBR** to locate the exact entry[cite: 5].
- The hardware then reads the PTE directly from RAM[cite: 5].

Step 2: Protection Checks and Data Access (Slide 15)

```
c

// 4. Check if the page is allocated and valid
if (PTE.Valid == False) { //[cite: 5]
    RaiseException(SEGMENTATION_FAULT); //[cite: 5]
}
// 5. Check if the operation matches permission bits (Read/Write/Execute)
else if (CanAccess(PTE.ProtectBits) == False) { //[cite: 5]
    RaiseException(PROTECTION_FAULT); //[cite: 5]
}
else {
    // 6. Access is permitted: extract offset and form physical address
    offset    = VirtualAddress & OFFSET_MASK; //[cite: 5]
    PhysAddr = (PTE.PFN << PFN_SHIFT) | offset; //[cite: 5]

    // 7. Access actual data in RAM (Second memory access!)
    Register = AccessMemory(PhysAddr); //[cite: 5]
}
```

- If `Valid == False`, the process attempted to dereference an unallocated memory address; the CPU aborts with a **Segmentation Fault**[cite: 5].
- If permissions don't match (e.g., writing to a Read-Only Code page), the CPU raises a **Protection Fault**[cite: 5].
- If all checks pass, the MMU combines the physical frame number (`PFN`) with the unchanged `offset` to generate the physical address, finally accessing the target data in RAM[cite: 5].

Detailed Memory Trace of an Array Loop (Slides 16 & 17)

Let's examine a concrete code example to trace how many memory accesses occur during program execution[cite: 5]:

```
c

int array[1000];
for (i = 0; i < 1000; i++) {
    array[i] = 0; //[cite: 5]
}
```

When compiled with optimization (`gcc -O`), the inner loop produces four assembly instructions[cite: 5]:

assembly

```

0x1024 movl $0x0, (%edi,%eax,4) ; array[i] = 0 (store 0 at edi + eax*4)
0x1028 incl %eax                ; i++ (increment index counter)
0x102c cmpl $0x03e8, %eax       ; compare i with 1000 (0x03e8 = 1000)
0x1030 jne 0x1024               ; if i != 1000, jump back to loop start

```

- Register `%edi` holds the base address of `array` (starting at virtual address `40000`) [cite: 5].
- Register `%eax` holds loop index `i` (scaled by 4 because each integer is 4 bytes) [cite: 5].
- Code instructions live on a code page around virtual address `1024` [cite: 5].

How Many Memory Accesses Occur in a Single Loop Iteration? (Slide 17)

For **one single iteration** of this 4-instruction loop, tracing every instruction fetch and memory access demonstrates the overhead [cite: 5]:

Iteration Step-by-Step Breakdown:

1. Fetch instruction ``movl`` at VA `0x1024`:
 - Access 1: Read PTE for Code (Page Table[1]) from RAM [cite: 5]
 - Access 2: Read instruction ``movl`` from Code physical address [cite: 5]
2. Execute instruction ``movl`` (Store 0 to ``array[i]`` at VA `40000`):
 - Access 3: Read PTE for Data (Page Table[39]) from RAM [cite: 5]
 - Access 4: Write 0 into array physical address [cite: 5]
3. Fetch instruction ``incl`` at VA `0x1028`:
 - Access 5: Read PTE for Code (Page Table[1]) from RAM [cite: 5]
 - Access 6: Read instruction ``incl`` from Code physical address [cite: 5]
 - (Execute ``incl``: Register-only increment, 0 memory accesses) [cite: 5]
4. Fetch instruction ``cmpl`` at VA `0x102c`:
 - Access 7: Read PTE for Code (Page Table[1]) from RAM [cite: 5]
 - Access 8: Read instruction ``cmpl`` from Code physical address [cite: 5]
 - (Execute ``cmpl``: Register-only compare, 0 memory accesses) [cite: 5]
5. Fetch instruction ``jne`` at VA `0x1030`:
 - Access 9: Read PTE for Code (Page Table[1]) from RAM [cite: 5]
 - Access 10: Read instruction ``jne`` from Code physical address [cite: 5]
 - (Execute ``jne``: Register jump, 0 memory accesses) [cite: 5]

The Bottleneck:

To execute this simple 4-instruction loop iteration, the CPU had to perform **10 separate physical memory accesses** [cite: 5]!

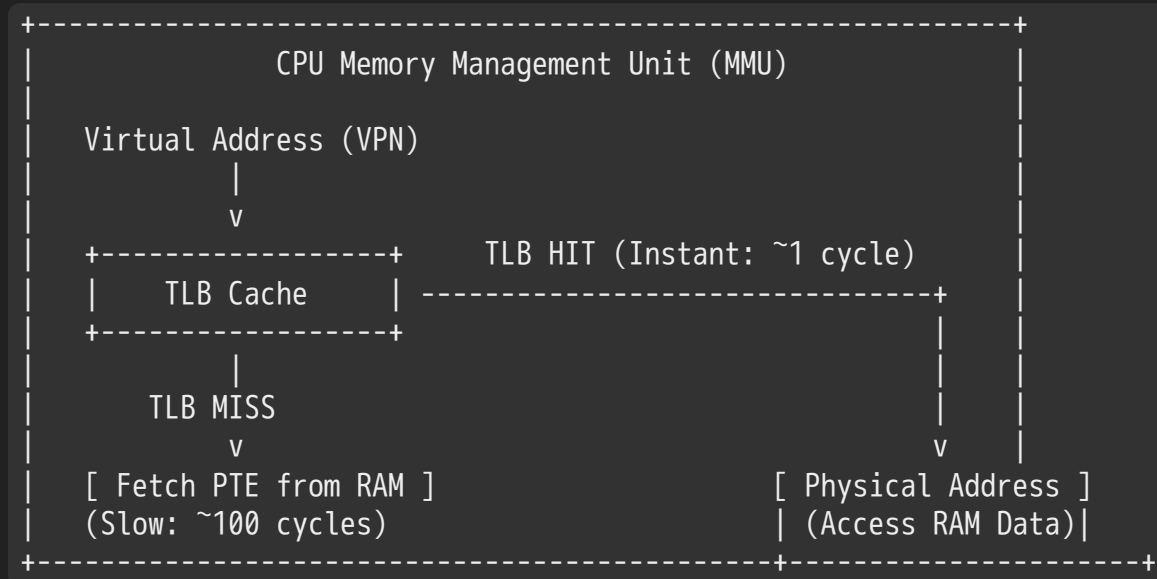
- 5 accesses were just reading Page Table Entries from RAM [cite: 5]!
- Notice the repeated accesses: **Page Table[1]** (the PTE for the code segment) was fetched from RAM **4 separate times** in a single iteration [cite: 5]!

The Solution: Caching Translations via the TLB (Slide 17 Annotations)

The handwritten notes on Slide 17 point out the engineering solution[cite: 5]:

"Fetch repeated PTEs from RAM every time? Too slow. We must use a hardware cache[cite: 5]!"

Instead of reading the page table in RAM on every instruction, modern CPUs include a high-speed on-chip hardware cache called the **TLB (Translation Lookaside Buffer)**[cite: 5].



- **TLB Hit:** When the CPU translates a VPN that was recently accessed, the translation is retrieved directly from the on-chip TLB in a single clock cycle, skipping the in-memory page table access entirely[cite: 5].
- **TLB Miss:** Only when a page is accessed for the first time does the MMU fetch the PTE from RAM, after which it caches the translation in the TLB for subsequent accesses[cite: 5].

This completes all slides, annotations, and trace examples for **Chapter 18: Paging: Introduction**[cite: 5].

Whenever you are ready, provide the PDF or name of your next chapter (likely Chapter 19: Paging: Faster Translations (TLB) or Multi-level Page Tables), and we will begin with Lesson 1.